



Московский государственный университет имени
М.В.Ломоносова
Факультет вычислительной математики и кибернетики
Кафедра автоматизации систем вычислительных комплексов

Рогинко Алла Дмитриевна

**Исследование и реализация жадных
алгоритмов
для задачи распределения таблиц
классификации
по стадиям конвейера СПУ**

Курсовая работа

Научный руководитель:
Доцент, к.ф.-м.н.
Капитонова Алла Петровна
Научный консультант:
Никифоров Никита Игоревич

Москва, 2026

Аннотация

Исследование и реализация жадных алгоритмов
для задачи распределения таблиц классификации
по стадиям конвейера СПУ

Рогинко Алла Дмитриевна

Рассматривается задача размещения логических match-action таблиц P4 по стадиям конвейера архитектуры RMT при учёте графа зависимостей таблиц и ограничений аппаратных ресурсов. Проводится обзор жадных эвристик (FFD, FFL, FFLS, MCF), описывается декомпозиция задачи на этапы топологической упорядоченности, проверки ресурсов и поиска размещения. Представлены программная реализация многопроходного варианта FFD с поддержкой горизонтального разрезания таблиц и эталонной реализации FFLS, а также методика экспериментального сравнения по числу занятых стадий и времени работы на синтетических конфигурациях. Сформулированы выводы о применимости рассмотренных подходов при различных размерах и плотности зависимостей графа.

Abstract

Содержание

1	Введение	5
2	Предметная область	6
2.1	Сетевые процессорные устройства	6
2.2	Архитектура RMT	6
2.3	Программа обработки пакетов и язык P4	7
2.4	Граф зависимостей и виды зависимостей внутри таблиц P4	7
3	Цель работы и постановка задачи	9
3.1	Актуальность работы	9
3.2	Цель работы	9
3.3	Постановка задачи	9
3.4	Ограничения	10
4	Обзор существующих алгоритмов	12
4.1	Задача размещения в контексте компиляции P4	12
4.2	Жадные эвристики размещения	12
4.3	Декомпозиция задачи на подзадачи	13
5	Реализованные алгоритмы	15
5.1	Горизонтальное разрезание таблиц	15
5.2	Многопроходный FFD	15
5.3	FFLS	16
6	Описание архитектуры реализованных алгоритмов	17
6.1	Структура кода и основные модули	17
7	Экспериментальное исследование	19
7.1	Методика экспериментального исследования	19
7.2	Анализ полученных результатов	19
7.2.1	Результаты сравнения алгоритмов по времени выполнения	19
7.2.2	Результаты сравнения алгоритмов по количеству стадий . .	19

8 Заключение	21
Список литературы	22

1 Введение

В современных сетях возникает необходимость в гибкой обработке пакетов. Необходимо быстро внедрять новые протоколы и сервисы без замены оборудования. Решением этого вопроса является использование программируемых сетевых устройств с архитектурой RMT с языком описания пакетной обработки P4. Такие устройства реализуют конвейерную обработку, состоящую из последовательных match-action стадий.

При компиляции P4-программы для архитектуры RMT нужно распределить логические таблицы по стадиям конвейера. От этого решения зависят число задействованных стадий и, следовательно, задержка обработки пакета. Для успешного размещения следует учитывать ограничения архитектуры аппаратуры и порядок применения таблиц, заданный графом зависимостей (Table Dependency Graph). На практике стремятся минимизировать число занятых стадий, поскольку оно непосредственно определяет задержку обработки пакета.

В данной работе рассматриваются жадные алгоритмы решения задачи размещения, анализируется их эффективность и быстрота выполнения, и предлагается модификация с поддержкой горизонтального разрезания таблиц.

2 Предметная область

2.1 Сетевые процессорные устройства

Сетевые процессорные устройства (СПУ) — это специализированные системы, предназначенные для обработки сетевого трафика на высоких скоростях. Проще говоря, такие устройства должны очень быстро принимать пакеты, анализировать их и принимать решение о дальнейшей обработке. В отличие от традиционных сетевых устройств, где логика работы часто жёстко зафиксирована на уровне аппаратуры, программируемые СПУ позволяют изменять алгоритмы обработки пакетов и адаптировать их под разные задачи. Отличительной чертой современных СПУ является наличие программируемой логики обработки пакетов, что позволяет адаптировать устройство под новые протоколы и сетевые функции без замены аппаратного обеспечения.

2.2 Архитектура RMT

Одной из архитектур эталонной для построения высокопроизводительных программируемых сетевых устройств является RMT (Reconfigurable Match Tables). Основная идея этой архитектуры заключается в использовании набора настраиваемых таблиц сопоставления (match tables), которые применяются на разных стадиях конвейера обработки пакетов. На каждой стадии на основе выбранного ключа выполняется поиск в одной или нескольких таблицах, где правила сопоставления определяют, какие действия должны быть выполнены над пакетом. Эти действия могут включать изменение заголовков, выбор следующей стадии обработки или передачу пакета на выходной интерфейс. Благодаря такой структуре архитектура остаётся гибкой и позволяет реализовывать различные алгоритмы обработки сетевого трафика.

Match-action стадии — последовательность одинаковых стадий. Каждая стадия содержит:

1. Таблицы точного поиска, реализованные на SRAM.
2. Таблицы тернарного и префиксного поиска, реализованные на TCAM.

3. Кроссбары для подачи произвольных полей из PHV на вход таблиц.
4. Action-блоки для параллельного изменения полей PHV.

2.3 Программа обработки пакетов и язык P4

Для описания поведения программируемого коммутатора используется язык P4 (Programming Protocol-independent Packet Processors). Программа на P4 определяет набор логических match-action таблиц, каждая из которых включает набор полей заголовка, по которым производится поиск и список возможных действий для них. P4 является аппаратно-независимым языком: одна и та же программа может компилироваться для различных устройств.

2.4 Граф зависимостей и виды зависимостей внутри таблиц P4

При компиляции P4-программы в конвейер RMT строится граф зависимостей таблиц (Table Dependency Graph, TDG). Вершинами графа являются логические таблицы, а рёбрами — зависимости между таблицами. Корректное размещение таблиц по стадиям конвейера должно удовлетворять всем видам зависимостей, иначе обработка пакетов будет нарушена. Выделяют четыре основных типа зависимостей:

Match-зависимость возникает, когда таблица A изменяет поле, а таблица B выполняет поиск по этому полю. Эта зависимость соответствует классической зависимости «чтение после записи». Для корректной работы необходимо, чтобы A полностью завершила выполнение своих действий до того, как B начнёт поиск. Поэтому, таблицы с match-зависимостью не могут располагаться в одной стадии конвейера.

Action-зависимость появляется, когда две таблицы A и B изменяют одно и то же поле, и важен результат, полученный в таблице B . При размещении допускается нахождение A и B в одной стадии.

Successor-зависимость заключается в следующем: выполнение таблицы B зависит от результата таблицы A . Таблицы с successor-зависимостью могут размещаться в одной стадии.

Обратная match-зависимость возникает, когда таблица A выполняет поиск по полю, которое впоследствии изменяется таблицей B . Также требует строгого разделения стадий: A должна быть размещена в стадии, строго меньшей, чем B .

3 Цель работы и постановка задачи

3.1 Актуальность работы

Рост скорости сетевого трафика и требование быстро менять поведение коммутаторов без замены оборудования делают актуальной программируемую сетевую обработку. Компиляция Р4-программы в конвейер RMT включает этап размещения логических таблиц по стадиям при ограничениях памяти и графе зависимостей. Качество этого этапа влияет на число занятых стадий и задержку обработки пакета, поэтому исследование эвристик размещения и их реализации для типовых ограничений TDG сохраняет практический интерес для разработчиков компиляторов и сетевого оборудования.

3.2 Цель работы

Целью данной работы является исследование жадных алгоритмов решения задачи размещения логических таблиц классификации по стадиям конвейера RMT-коммутатора, а также разработка программной реализации этих алгоритмов для оценки ресурсных затрат Р4-программ.

3.3 Постановка задачи

Формально задача размещения таблиц классификации (pipeline embedding problem) может быть сформулирована следующим образом.

Дано:

1. Множество логических таблиц $T = \{t_1, t_2, \dots, t_n\}$, каждая из которых характеризуется:
 - типом сопоставления $\text{match_type}(t_i) \in \{\text{exact}, \text{ternary}, \text{lpm}\}$;
 - шириной ключа $w(t_i)$ (в битах);
 - максимальным количеством записей $e(t_i)$;
 - списком зависимостей от других таблиц с указанием типа зависимости $\text{dep_type} \in \{\text{match}, \text{action}, \text{successor}, \text{reverse_match}\}$.

2. Описание аппаратной архитектуры RMT:

- количество match-action стадий S_{total} ;
- количество блоков SRAM на стадию B_{sram} , ширина блока W_{sram} (бит), глубина блока D_{sram} (количество записей);
- количество блоков TCAM на стадию B_{tcam} , ширина блока W_{tcam} (бит), глубина блока D_{tcam} (количество записей).

Требуется: Построить отображение $\phi : T \rightarrow \mathbb{N}$ (номер стадии для каждой таблицы), а в случае горизонтального разрезания — распределение фрагментов таблицы по нескольким стадиям, такое, что:

1. Выполнены все аппаратные ограничения (ёмкость памяти, совместимость типов).
2. Соблюдены все типы зависимостей между таблицами.
3. Количество использованных стадий $\max_{t \in T} \phi(t) - \min_{t \in T} \phi(t) + 1$ минимально.

3.4 Ограничения

В рамках данной работы учитываются следующие ограничения, вытекающие из архитектуры RMT и свойств P4-программ:

Ограничение размещения — каждая логическая таблица должна быть размещена хотя бы в одной физической стадии конвейера (возможно, с разбиением на фрагменты).

Ограничение ёмкости — суммарное количество блоков памяти (SRAM или TCAM), требуемое для всех таблиц, размещённых в одной стадии, не может превышать доступного количества блоков этого типа в стадии.

Ограничение совместимости — таблицы с типом сопоставления `ternary` или `lpm` могут использовать только TCAM; таблицы с типом `exact` могут использовать как SRAM, так и TCAM (но предпочтительнее SRAM в силу энергоэффективности).

Ограничение зависимостей — для каждой пары таблиц (t_i, t_j) , связанных зависимостью, должно выполняться:

- для `match` и `reverse_match`: $\phi(t_i) < \phi(t_j)$ (строгое разделение);
- для `action` и `successor`: $\phi(t_i) \leq \phi(t_j)$ (допускается одна стадия).

Ограничение последовательности фрагментов — при горизонтальном разрезании таблицы все её фрагменты должны располагаться в стадиях с номерами, образующими строго возрастающую последовательность без пропусков: $s_1 < s_2 < \dots < s_k$ и $s_{i+1} \leq s_i + 1$.

Ограничение целостности match-зависимости при разрезании — если таблица t_i (предок) разрезана на фрагменты, а таблица t_j (потомок) связана с t_i match-зависимостью, то все фрагменты t_i должны быть размещены в стадиях, строго меньших, чем все фрагменты t_j : $\max \phi_{\text{frag}}(t_i) < \min \phi_{\text{frag}}(t_j)$.

4 Обзор существующих алгоритмов

4.1 Задача размещения в контексте компиляции R4

Задача отображения логических таблиц на физические стадии RMT относится к классу задач составления расписаний на параллельных конвейерах с ограниченными ресурсами каждого шага и предшествованиями между работами [1]. Если абстрагироваться от деталей конкретной микросхемы, каждую таблицу естественно рассматривать как «работу» с требованиями к памяти (SRAM или TCAM) и ширине ключа; стадию — как «машину» с ограниченной суммарной вместимостью. При этом орграф зависимостей TDG задаёт частичный порядок на множестве таблиц: некоторые пары таблиц обязаны оказаться на разных стадиях (жёсткие предшественники), другие могут выполняться на одном такте конвейера.

Исследование и разработку алгоритмов размещения для программируемых коммутаторов ведут как производители аппаратуры, так и сообщество открытых компиляторов; публикации описывают как точные методы для малых экземпляров, так и масштабируемые эвристики, пригодные для промышленных R4-программ [2]. На практике полный перебор и целочисленное программирование малой размерности применимы ограниченно, что стимулирует использование жадных приближённых методов и локального поиска.

4.2 Жадные эвристики размещения

Общая схема жадного размещения следующая: таблицы обходятся в заранее зафиксированном порядке; для каждой таблицы выбиется первая (в смысле выбранного правила) стадия, на которой одновременно выполняются условия (1) корректности относительно TDG, (2) достаточности свободных ресурсов. Отличие эвристик заключается в порядке обхода и в правилах разрешения узких мест.

FFD (First Fit Decreasing). Перед размещением таблицы упорядочивают по убыванию объёма требуемых ресурсов (или по числу занимаемых единиц TCAM или SRAM—в зависимости от того, какой ресурс является дефицитным).

Затем каждая таблица первым делом ставится на минимально возможный номер стадии среди допустимых. Интуитивно, «тяжёлые» таблицы резервируют ранние стадии, что уменьшает риск фрагментации ресурсов на поздних тактах. Недостаток чистого FFD — чувствительность к порядку при равных размерах и отсутствие явного учёта глубины в TDG.

FFL (First Fit by Level). Порядок обхода определяется топологическим «уровнем»: сначала размещают таблицы без предшественников (или с минимальной длиной longest path от истоков), затем следующий слой и т. д. Внутри уровня возможны дополнительные критерии (например, убывание размера). Такой порядок согласует жадность с направлением потока данных в графе и снижает вероятность раннего «закрытия» всех допустимых стадий для потомков.

FFLS (First Fit by Level and Size). Комбинирует идеи FFL и FFD: на каждом топологическом уровне таблицы упорядочивают по убыванию ресурсоёмкости, после чего применяют правило первого подходящего fit. Алгоритм часто используется как понятный и быстрый базовый ориентир при сравнении с более сложными схемами.

MCF и родственные модификации. В ряде работ рассматривают эвристики, явно минимизирующие на каждом шаге «стоимость» выбора стадии (например, Marginal Cost Fit — приращение загрузки ресурсов или штраф за увеличение глубины конвейера). Формально MCF требует вычислять для кандидатной стадии приращение целевой функции; на практике это даёт дополнительный вычислительный overhead, зато иногда улучшает качество на графах с неравномерной плотностью зависимостей.

При разработке реализации идею горизонтального разрезания («сплита») крупной таблицы на фрагменты при дефиците ресурсов на одном такте конвейера используют в связке с жадными проходами; конкретный механизм и его встраивание в многопроходный FFD описаны в разделе 5.1.

4.3 Декомпозиция задачи на подзадачи

Исходная задача «найти размещение с минимальным числом стадий при соблюдении TDG и лимитов ресурсов» может быть разложена на цепочку более простых шагов, каждый из которых либо решается прямым алгоритмом, либо

сводится к известной подзадаче.

1. **Построение и классификация рёбер TDG.** На входе задаётся множество таблиц и отношения четырёх типов (match, action, successor, reverse match). Рёбра с ограничением «нельзя в одной стадии» порождают дополнительные условия на разницу номеров стадий; остальные задают частичный порядок или совместимость на одном такте. Результат — единый ориентированный граф зависимостей с пометками типов.
2. **Топологическая стратификация.** Вычисление уровней (или длин критического пути к каждой вершине) даёт приоритеты для обхода в FFL/FFLS и используется для проверки отсутствия циклов, разрушающих конвейерную семантику.
3. **Проверка допустимости стадии для таблицы.** Для фиксированной пары (таблица, номер стадии) проверяются: (а) все предшественники по TDG размещены не позже (или строго раньше — по типу зависимости); (б) суммарная занятость SRAM/TCAM на стадии после добавления таблицы не превышает квот; (в) совместимость с уже размещёнными на этой стадии таблицами с точки зрения match и обратных match-зависимостей.
4. **Жадный выбор стадии.** Подзадача сводится к перебору стадий в возрастающем порядке с ранним выходом при первом успехе (first fit) либо к выбору стадии по минимальному приращению стоимости (MCF).
5. **Обработка переполнения.** Если подходящей стадии нет, вызывается процедура горизонтального разрезания; для каждого фрагмента подзадачи (3)–(4) повторяются. Терминация гарантируется при наличии конечного числа допустимых сплитов и стадий.
6. **Оценка качества.** Число использованных стадий, максимальная загрузка ресурсов и время работы алгоритма сравниваются между эвристиками на фиксированном наборе входов.

Такая декомпозиция отделяет чисто графовые аспекты от учёта аппаратных квот и позволяет независимо тестировать корректность проверки зависимостей и эффективность жадного размещения.

5 Реализованные алгоритмы

5.1 Горизонтальное разрезание таблиц

Если таблица не помещается целиком ни на одной допустимой стадии, программа размещения может разбить её на несколько фрагментов с последующим согласованным выполнением (*horizontal table split*). Это увеличивает число узлов TDG и усложняет проверку зависимостей, но позволяет избежать тупика при жёстких лимитах SRAM/TCAM. В реализации модуль горизонтального разрезания вызывается планировщиком, когда ни одна допустимая по TDG стадия не содержит достаточной остаточной квоты памяти при выполнении предиката допустимости. Сплиттер порождает фрагменты с обновлёнными требованиями ресурсов и дополнительными ограничениями предшествования так, чтобы части одной логической таблицы выполнялись последовательно с согласованием ключей; политика дробления задаётся конфигурацией и не затрагивает интерфейс остальных модулей (см. раздел 6).

5.2 Многопроходный FFD

Многопроходная реализация FFD строится как управляющий цикл над одношаговым размещением таблицы. Перед каждым проходом таблицы упорядочиваются по убыванию выбранной меры ресурсоёмкости; для текущей таблицы последовательно перебираются номера стадий, начиная с минимально допустимого согласно TDG. Для пары (*таблица, стадия*) проверяются: размещение всех предшественников и типы зависимостей (включая ограничения «не в одной стадии» для *match* и обратной *match*-связи), совместимость с уже размещёнными на стадии таблицами, суммарная занятость SRAM/TCAM. При успехе таблица фиксируется; при неудаче после полного перебора стадий либо увеличивается число стадий-приёмников ресурсов, либо выполняется разрезание (см. п. 5.1). После завершения прохода выполняется анализ таблиц, оставшихся без размещения: при наличии таковых возможны следующий проход с обновлённым графом (после сплитов) или изменение параметров упорядочивания вплоть до исчерпания лимита итераций. Таким образом, многопроходность охватывает повторную

подачу множества таблиц после частичных решений или дробления до стабилизации результата.

5.3 FFLS

Алгоритм FFLS (First Fit by Level and Size) в реализации сочетает топологический порядок с жадным first fit по стадиям. На входе вычисляются уровни вершин TDG (или эквивалентная топологическая стратификация); таблицы внутри каждого уровня упорядочиваются по убыванию ресурсоёмкости, после чего к каждой применяется тот же предикат допустимости и правило первой подходящей стадии, что и в FFD. Такой порядок согласует обход графа с направлением зависимостей и задаёт понятный эталон для сравнения с FFD без явного включения топологических уровней в правило упорядочивания последнего.

6 Описание архитектуры реализованных алгоритмов

6.1 Структура кода и основные модули

Для прототипирования алгоритмов размещения выбран язык Python 3: удобны графовые структуры, словари описаний таблиц и воспроизводимость экспериментов; перенос критичных по времени участков на компилируемый язык возможен без изменения логики размещения.

Для представления графа зависимостей и вычисления уровней используются структуры на основе стандартной библиотеки (`dict` списков смежности, очередь при обходе в ширину). Зависимость от сторонних пакетов минимизирована; при необходимости визуализации TDG допускается подключение `networkx`.

Компоненты комплекса. Логическая структура следующая:

1. модуль входных данных — загрузка описания набора таблиц, ключей и требований к SRAM/TCAM, списков рёбер TDG по четырём категориям зависимостей;
2. модуль валидации — проверка согласованности графа (отсутствие конвейерно неприемлемых циклов и пр.);
3. планировщик — реализации FFD с многопроходным режимом и FFLS (и опционально FFL/MCF для сравнения) через общий интерфейс *граф ресурсов* $\rightarrow$ *расписание по стадиям*;
4. модуль горизонтального разрезания (сплиттер) — см. раздел 5.1;
5. модуль отчёта — число стадий, профиль загрузки SRAM/TCAM и замеры времени.

Уровни абстракции. На каноническом уровне TDG задаётся в терминах логических таблиц P4 и классифицированных зависимостей; на планировочном — таблицы снабжаются числовой ресурсоёмкостью и упорядочиваются

выбранной эвристикой; на размещающем ведётся состояние конвейера: остаточные квоты и множества размещённых таблиц на стадии, операции допустимости, коммита и при необходимости отката.

Разделение ответственности. Модуль графовых операций инкапсулирует уровни (длину пути для FFL/FFLS), списки предшественников и классификацию ограничений «нельзя в одной стадии». Модуль ресурсов суммирует SRAM/TCAM и проверяет лимиты с учётом типа таблицы. Ядро FFD включает сортировку по убыванию веса; ядро FFLS вызывает сортировку внутри слоёв топологической декомпозиции. Многоходовой FFD со сплитами реализован как внешний цикл над одношаговым разместителем (согласование с разделом 5).

Интерфейсы эвристик. Эвристики реализуют общий метод `place(tdg, limits)` с отображением «таблица $\rightarrow$ номер стадии» и диагностикой (число итераций, число сплитов); добавление MCF или локального улучшающего прохода сводится к тому же интерфейсу. Лимиты стадий и квот памяти вынесены в отдельный объект для эмуляции разных целевых профилей RMT.

Сложность и время. Пусть n — число логических таблиц после возможных сплитов, m — верхняя граница числа стадий, d — среднее число зависимостей на таблицу. Проверка допустимости для пары (таблица, стадия) в наивной реализации — $O(d)$ к предшественникам и $O(k)$ конфликтов на стадии ($k \leq n$); перебор стадий даёт $O(m)$, размещение одной таблицы в худшем случае $O(m(d+k))$, для всего набора порядка $O(nm(d+n))$, что при $n, m \ll 10^3$ допускает интерактивные эксперименты. Память: $O(n+d)$ на граф, $O(m)$ на размещение и остатки ресурсов, $O(n)$ на вспомогательные структуры уровней.

На синтетических конфигурациях с сотнями таблиц время одного запуска, как правило, остаётся долями секунды; узким местом становятся генерация очень плотных TDG или полное трассирование решений.

7 Экспериментальное исследование

7.1 Методика экспериментального исследования

Цель экспериментов — сравнить жадные эвристики FFD с многопроходным режимом и горизонтальным разрезанием и FFLS по метрикам: (1) число задействованных стадий конвейера после успешного размещения; (2) максимальная и средняя утилизация SRAM и TCAM по стадиям; (3) процессорное время работы размещителя на одном экземпляре TDG. Чем меньше число стадий при соблюдении зависимостей и ограничений, тем ниже модельная задержка обработки пакета; время выполнения алгоритма важно при инкрементальной компиляции крупных программ.

Формируются синтетические TDG с контролируемыми параметрами: число вершин n , средняя исходящая степень, доля рёбер каждого типа зависимостей (match, action, successor, reverse match), диапазоны требований к памяти. Генератор исключает циклы, несовместимые с конвейерной моделью, и задаёт верхнюю границу числа стадий для условий «напряжённого» размещения. Для каждой тройки (n , плотность, соотношение типов) выполняется серия из k экземпляров (например, $k = 30$) с фиксированными зёрнами ГПСЧ. На каждом экземпляре запускаются оба алгоритма при идентичных лимитах ресурсов.

7.2 Анализ полученных результатов

7.2.1 Результаты сравнения алгоритмов по времени выполнения

Отношение времени FFLS к времени FFD на общих входах зависит от затрат на вычисление уровней и сортировок внутри слоёв: при малых n различия малы; при n порядка сотен дополнительные расходы FFLS остаются сортировочно-линейными и обычно не выходят за доли секунды в интерпретируемой реализации.

7.2.2 Результаты сравнения алгоритмов по количеству стадий

При низкой плотности TDG и умеренных квотах FFD и FFLS, как правило, дают близкое число стадий. С ростом плотности match и обратных match-

ограничений FFLS часто даёт меньше конфликтных попыток благодаря согласованию обхода с направлением зависимостей в TDG; при дефиците TCAM на ранних уровнях преимуществом может обладать FFD из-за раннего размещения крупных таблиц.

Многопроходный режим и сплиты заметнее снижают число стадий там, где встречаются редкие, очень объёмные таблицы; при этом возрастают время работы программы размещения и сложность представления решения из-за увеличения числа узлов после разрезания.

Подзадачи, сформулированные в разделе о постановке задачи, по сравнению эвристик и оценке влияния сплитов поддержаны воспроизводимой схемой бенчмаркинга; конкретные таблицы и графики целесообразно зафиксировать после финализации параметров генератора и целевого профиля RMT.

8 Заключение

В работе рассмотрена задача размещения логических match-action таблиц P4 по стадиям конвейера архитектуры RMT при учёте графа зависимостей и аппаратных лимитов памяти.

Сформулированы цель и научно-технические задачи по анализу жадных эвристик FFD, FFL, FFLS и MCF, разработке многопроходного FFD с поддержкой горизонтального разрезания таблиц, реализации FFLS как эталона сравнения, учёту четырёх типов зависимостей в TDG и экспериментальному сравнению по числу стадий и времени работы.

В обзорном разделе показана связь задачи с расписанием на конвейере с ограниченными ресурсами, описаны свойства рассматриваемых жадных методов и выполнена явная декомпозиция исходной проблемы на подзадачи построения TDG, топологической стратификации, проверки допустимости стадии, жадного выбора и обработки переполнения.

Предложено программное решение на языке Python с модульной архитектурой (ввод и валидация TDG, ресурсный учёт, ядра эвристик, сплиттер, отчётность). Оценён порядок асимптотической сложности наивной реализации проверок размещения.

Описана методика синтетических экспериментов и качественно обсуждены ожидаемые различия между FFD и FFLS в зависимости от плотности и типов ограничений в графе.

Практическая значимость работы заключается в том, что построенная схема позволяет систематически сравнивать жадные алгоритмы размещения при проектировании и настройке компиляционного конвейера для программируемых сетевых устройств. Направления дальнейшей работы включают привязку модели ресурсов к конкретной микроархитектуре целевого чипа и сравнение с промышленным компилятором на реальных P4-программах; пост-жадную оптимизацию (локальный перенос между стадиями, симулированный отжиг на окне узлов TDG, ограниченное целочисленное программирование на кластерах связанных таблиц); формализацию аппроксимационных свойств для упрощённых подклассов графов TDG для обоснования порядка обхода в схемах типа FFLS.

Список литературы

- [1] P4: Programming Protocol-Independent Packet Processors / Pat Bosshart, Dan Daly, Glen Gibb et al. // *ACM SIGCOMM Computer Communication Review*. — 2014. — Vol. 44, no. 3. — Pp. 87–95.
- [2] p4c: reference compiler for the P4 language. — GitHub. — дата обращения: 11.05.2026. <https://github.com/p4lang/p4c>.
- [3] P4 Language Specification. — P4.org. — дата обращения: 11.05.2026. <https://p4.org/specs/>.